Final Project: Food Inspections

Chicago & Dallas Food Establishment Inspections Pipeline

Student	Akshay Dnyaneshwar Govind
NUID	002509562
Course	DAMG 7370 — Designing Data Architecture for Business Intelligence
Institution	Northeastern University
Semester	Spring 2026
Date	April 21, 2026
GitHub	github.com/akshaygovindd/FoodLens-Databricks-Pipeline

Tech Stack

Databricks	PySpark	Delta Lake	Unity Catalog	Power BI

Abstract

This project implements a production-style, metadata-driven data engineering pipeline for food establishment inspection data from Chicago and Dallas. The pipeline ingests 387,000+ records from public open data APIs, processes them through a Medallion Architecture (Bronze → Silver → Gold) on Databricks with custom DQX data quality validation, and loads a Kimball-style star schema with SCD Type 2 for restaurant history tracking. Four Power BI dashboards deliver actionable public health analytics.

Part 1: Data Profiling and Source Analysis

1.1 Project Overview

This project analyzes food establishment inspection data from Chicago and Dallas with the goal of uncovering public health insights and improving transparency. The pipeline follows Medallion Architecture (Bronze → Silver → Gold) on Databricks using Delta Tables, with data quality validation through a custom DQX engine.

1.2 Source Datasets

Attribute	Chicago	Dallas
Source Format	CSV	CSV
Row Count (Raw)	308,161	78,984
Inspection Identifier	Inspection_ID (numeric)	Derived via MD5 hash
Violations Structure	Single pipe-delimited text field	25 wide columns per inspection row
Violation Scoring	Derived from Results (Pass=90, Fail=70, etc.)	Numeric score (0–100)
Restaurant Identifier	MD5(DBA + facility + address + zip)	MD5(name + address + zip)
Risk Category	Embedded in Risk field e.g. Risk 1 (High)	Not provided — defaulted to 'Unknown'
Facility Type	Provided directly	Derived via keyword matching on DBA name
State	Provided (IL)	Derived (TX)
Address	Single Address field	Split across Street_Number, Street_Name, Street_Direction, Street_Type, Street_Unit
Lat/Long	Separate columns	Embedded in Lat_Long_Location string

1.3 Schema Differences and Harmonization Strategy

- Chicago violations stored as a single pipe-delimited string — parsed using regex to extract violation code, description, and inspector comments.
- Dallas violations stored in up to 25 wide columns per inspection row — unpivoted into one row per violation using a slot-based union pattern.
- Both datasets standardized to a common SILVER_COLS schema: one row per violation per inspection.
- Violation IDs generated via MD5 hash of violation_code, violation_description, and violation_detail.
- Restaurant IDs generated via MD5 hash of DBA_Name, facility_type, address, and zip_code.
- Inspection IDs for Dallas derived via MD5 of DBA_Name, inspection_date, address, and zip_code.

1.4 Data Profiling Findings

Chicago

- 308,161 raw rows ingested into bronze_chicago.
- Violations stored as unstructured text — required regex-based parsing.
- Results (Pass, Fail, Pass w/ Conditions, No Entry) mapped to numeric scores.
- is_critical_flag derived from keywords 'Urgent'/'Critical' in violation content.

Dallas

- 78,984 raw rows ingested into bronze_dallas.
- Wide violation schema with up to 25 violation slots per inspection.
- Facility type not explicitly provided — derived using keyword-matching logic on DBA_Name.
- Risk category not available — set to 'Unknown' for all Dallas records.
- Latitude/longitude embedded in a single string column — extracted via regex.
- DQX found 1,300 bad rows: 1,186 invalid zip codes, 75 null DBA names, 39 out-of-range scores.

Part 2: Dimensional Model Design

2.1 Modeling Approach

The dimensional model is a star schema designed to support analytical queries across inspection results, violation details, restaurant characteristics, and time. The model consists of two fact tables and three dimension tables stored in the Gold Zone.

2.2 Dimensional Model Overview

Table	Type	Grain
dim_date	Dimension	One row per unique inspection date
dim_restaurant	SCD Type 2	One row per version of a restaurant entity
dim_violation_detail	Dimension	One row per unique violation code/description combination
fact_inspections	Fact	One row per inspection event
fact_violations	Fact	One row per violation per inspection

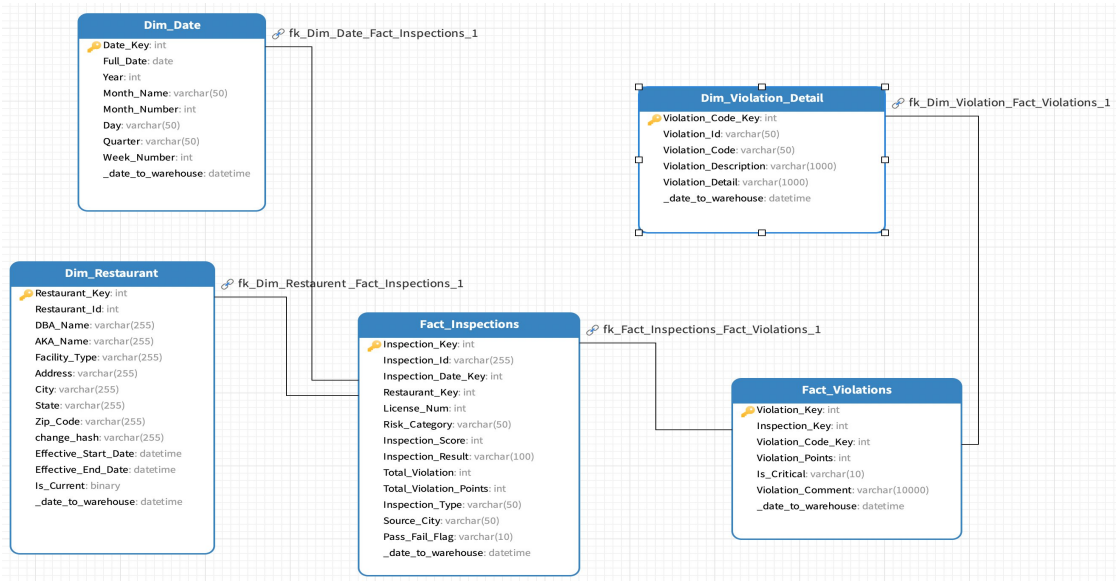

Figure 1: Star Schema — Gold Zone Dimensional Model (ER Studio)

2.3 Table Definitions

dim_date

Standard date dimension from all distinct inspection dates in Silver. Attributes: year, month name, month number, quarter, day of week, week number. date_key formatted as YYYYMMDD integer for fast joins.

dim_violation_detail

Stores all unique violations across both cities. Each row identified by violation_id (MD5 of code + description + detail). Dallas and Chicago codes maintained separately but stored in the same dimension table.

dim_restaurant (SCD Type 2)

Tracks restaurant history with SCD Type 2 logic. change_hash (MD5 of AKA_Name) detects changes. On change: existing record expired (is_current=false), new active record inserted (effective_end_date=9999-12-31, is_current=true).

fact_inspections

One row per inspection event. Joins to dim_restaurant via restaurant_key and dim_date via inspection_date_key. Metrics: inspection_score, total_violation, total_violation_points, pass_fail_flag.

fact_violations

One row per violation per inspection. Joins to fact_inspections via inspection_key and dim_violation_detail via violation_code_key. Attributes: violation_points, is_critical, violation_comment.

2.4 Foreign Key Relationships

Relationship	From Table	FK Column	To Table	PK Column
Inspection → Date	fact_inspections	inspection_date_key	dim_date	date_key
Inspection → Restaurant	fact_inspections	restaurant_key	dim_restaurant	restaurant_key
Violation → Inspection	fact_violations	inspection_key	fact_inspections	inspection_key
Violation → Violation Detail	fact_violations	violation_code_key	dim_violation_detail	violation_code_key

Part 3: Loading the Dimensional Model (Silver to Gold)

3.1 Pipeline Architecture

Notebook	Description
01_Environment_Setup	Creates catalog, schemas, volumes, metadata tables, and empty Gold layer DDL.
02_Raw_to_Bronze_Load	Reads CSVs from Raw Volume, writes timestamped Parquet to Staging Volume, loads Bronze Delta tables, logs
03_Bronze_to_Silver_Load	Transforms Bronze tables into unified Silver schema, applies DQX rules, enforces business rules, deduplicates v
04_Silver_to_Gold_Load	Reads silver_table and loads star schema via Delta MERGE: dim_date, dim_violation_detail, dim_restaurant (SC

3.2 Raw to Bronze

- Reads raw CSVs from the landing Volume using configurable path mappings (LANDING_FILE_MAP).
- Writes Parquet to Staging Volume with timestamped path: YYYY/MM/DD/filename_YYYYMMDD_HHMMSS.parquet.
- Row-count reconciliation: source vs. target count compared; status (SUCCESS/FAILED/SKIPPED) logged to child_metadata.
- Metadata columns added: _batch_id, _source_city, _ingested_at, _bronze_loaded_at.
- Dallas: 78,984 rows → bronze_dallas. Chicago: 308,161 rows → bronze_chicago.

3.3 Bronze to Silver

Chicago Transformation

- Violations parsed from pipe-delimited string using split and regex.
- Violation code extracted from numeric prefix of each segment (e.g. '36. DESCRIPTION').
- Results mapped to scores: Pass=90, Pass w/ Conditions=80, Fail=70, No Entry=0.
- is_critical_flag derived from keywords 'Urgent'/'Critical' in violation description and comments.

Dallas Transformation

- Column names normalized: Violation_Description_-_1 → violation_description_1.
- 25 wide violation columns unpivoted into one row per violation using slot-based union pattern.
- Violation code extracted from *NN prefix in each violation description field.
- facility_type derived via keyword matching on DBA_Name (e.g. 'starbucks' → Cafe).

DQX Validation Rules

Rule	City	Action
DBA_Name cannot be null or empty	Both	Drop bad row
inspection_date cannot be null	Both	Drop bad row
inspection_type cannot be null or empty	Both	Drop bad row
zip_code must match ^\d{5}$	Both	Drop bad row
violation_score must be 0–100	Dallas	Drop bad row
inspection_result cannot be null	Chicago	Drop bad row
PASS cannot have Critical/Urgent violation	Both	Drop inspection rows

Rule	City	Action
Score >= 90 cannot have > 3 violations	Dallas	Drop inspection rows
Duplicate violations deduplicated	Both	Keep distinct

DQX Results

- Dallas: 1,300 bad rows removed (1,186 invalid zip codes, 75 null DBA names, 39 out-of-range scores).
- Final Silver table: 1,179,288 rows, 27 columns.
- ALL CHECKS PASSED: 0 PASS+critical violations, 0 scores > 100, 0 invalid zip codes.

3.4 Silver to Gold

Table	MERGE Key	Final Row Count
dim_date	date_key	4,518
dim_violation_detail	violation_id	911
dim_restaurant (SCD2)	restaurant_id + is_current	43,677
fact_inspections	inspection_id (safe append)	279,471
fact_violations	inspection_key + violation_code_key	1,179,288

3.5 Quarantine Table

A quarantine table (Quarantine_Inspection) was implemented to capture inspection records that would have been duplicated when loading fact_inspections. Records are tagged with quarantine_category, failed_column, source_system, and pipeline_name for audit.

Part 4: BI Dashboards

4.1 Dashboard Overview

Four Power BI dashboards were built on the Gold Zone star schema using a dark theme. All visuals connect directly to fact_inspections, fact_violations, dim_restaurant, dim_date, and dim_violation_detail.

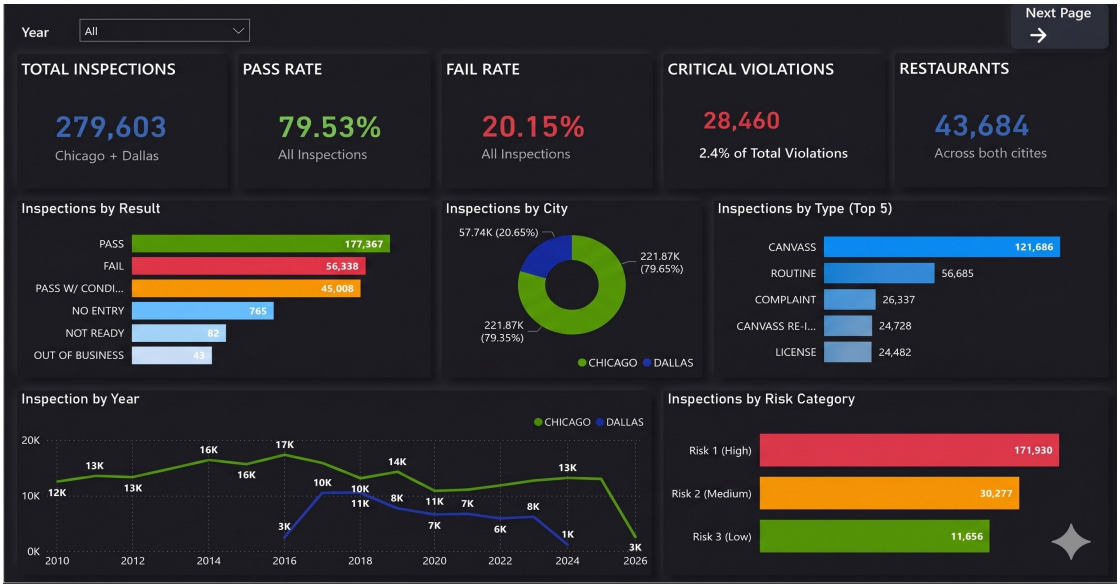

Figure 2: Overview Dashboard — 279,603 total inspections, 79.53% pass rate, 28,460 critical violations

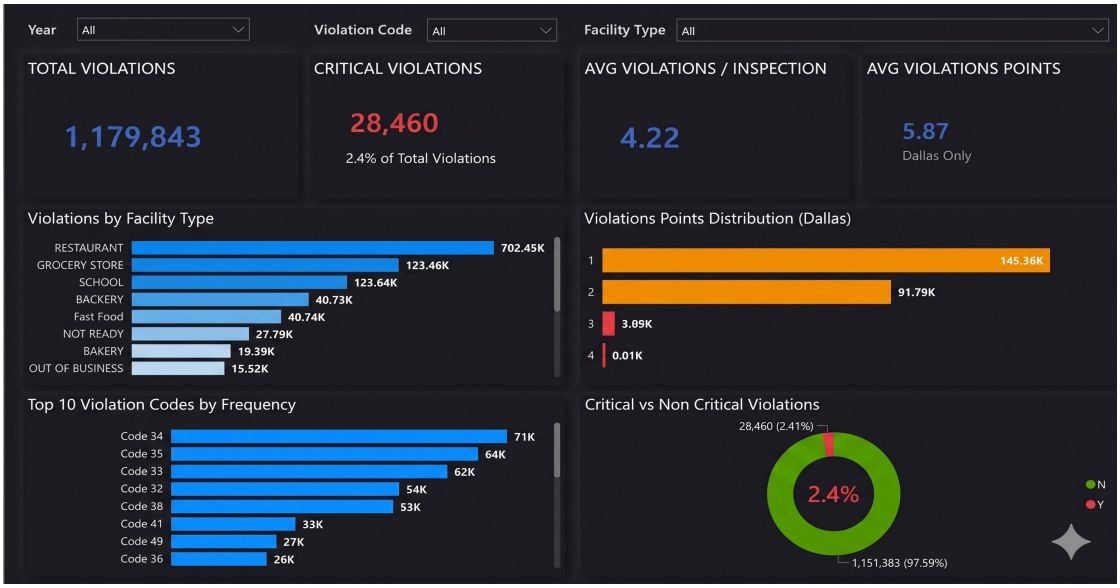

Figure 3: Violations Analysis — 1,179,843 total violations, Top 10 codes, facility type breakdown

Figure 4: Restaurant Analysis — 43,684 unique restaurants, 2,455 never passed, pass rate by facility type

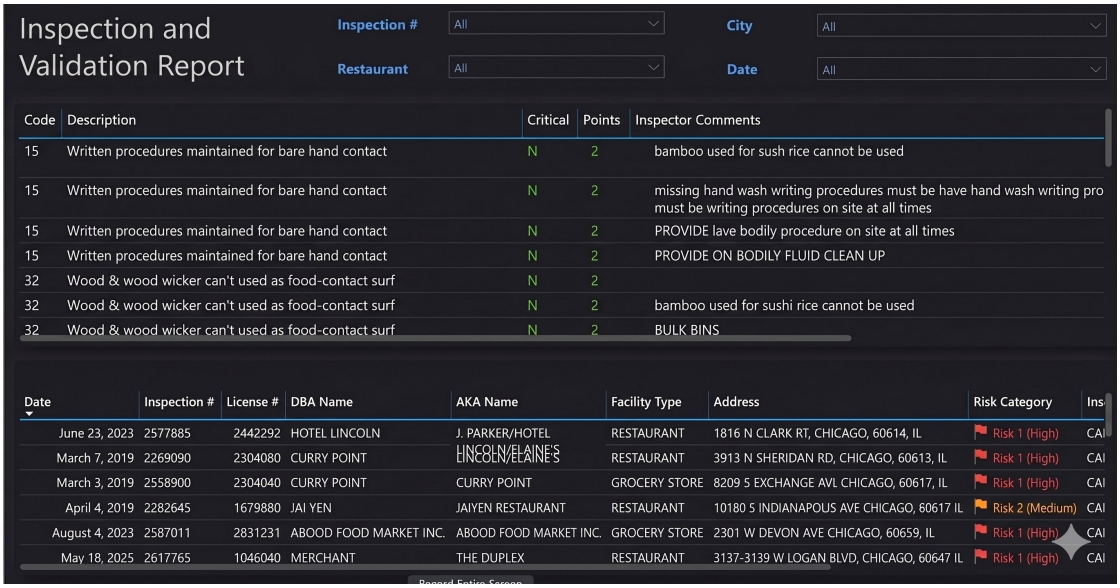

Figure 5: Inspection and Validation Report — drill-down with violation codes, points, inspector comments

Good Practices Incorporated

Practice	Implementation
Medallion Architecture	Raw Volume → Staging Volume (Parquet) → Bronze (Delta) → Silver (Delta) → Gold (Delta)
Metadata-Driven Pipeline	Parent metadata table drives active source selection. child_metadata logs run status, row counts, and batch IDs for every run.
Idempotent Loads	All Gold tables use Delta MERGE (not overwrite) — safe to rerun without data duplication.
Data Quality (DQX)	Custom DQX engine applies expectations at Silver layer. Bad rows tracked with error message column per rule.
SCD Type 2	dim_restaurant implements full SCD2 with change_hash detection, effective dates, and is_current flag.
MD5 Surrogate Keys	MD5 hashing for inspection_id, restaurant_id, and violation_id ensures consistent cross-city key generation.
Column Standardization	All column names lowercased, spaces replaced with underscores before Silver load.
Audit Columns	_date_to_warehouse, _batch_id, _ingested_at, _bronze_loaded_at added at every layer for lineage tracking.
Quarantine Pattern	Duplicate or invalid Gold-layer records routed to Quarantine_Inspection table with reason codes.
Parameterized Notebooks	No hardcoded values in any notebook — all constants centralized in Util notebook and passed at runtime.

Gold Layer Final Row Counts

Table	Type	Row Count
dim_date	Dimension	4,518
dim_restaurant	SCD Type 2	43,677
dim_violation_detail	Dimension	911
fact_inspections	Fact	279,471
fact_violations	Fact	1,179,288

Repository

All notebooks, dashboard screenshots, source-to-target mapping, and dimensional model are available at:

https://github.com/akshaygovindd/FoodLens-Databricks-Pipeline